Solucionario Docente: Patrones DAO/DTO y Pools HikariCP

Soluciones de referencia completas y documentadas para los ejercicios del Bloque 2

MÓDULO	UNIDAD	BLOQUE	PROFESOR
Acceso a Datos (0486)	UT2: JDBC y MySQL	B2: DAO, DTO y HikariCP	Sergio Capdevila Díez

Resultados de Aprendizaje: RA2 **Criterios de Evaluación:** `CE2.b` , `CE2.g` , `CE2.h`

Ejercicio 2.3: Configuración Avanzada de Pool HikariCP con Detección de Fugas

40 MIN

Configura un DataSource HikariCP en Java 21 estableciendo: `maximumPoolSize=5`, `connectionTimeout=10000`, y `leakDetectionThreshold=5000` (5 segundos). Desarrolla una prueba donde se obtiene una conexión y deliberadamente se olvida invocar `conn.close()`. Observa en los logs cómo HikariCP reporta una fuga de conexión indicando la línea exacta del código causante.

🔑 SOLUCIÓN COMPLETA Y CÓDIGO FUNCIONAL:

```
package es.cifpaviles.dam2.ad.ut2.b2;

import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class TestFugaConexionesHikari {
    public static void main(String[] args) throws Exception {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:mysql://localhost:3306/dam2_db?useSSL=false&allowPublicKeyRetrieval=true");
        config.setUsername("dam2_user");
        config.setPassword("dam2_pass");

        config.setMaximumPoolSize(5);
        config.setConnectionTimeout(5000);
        // Habilitar detección de fugas (avisa si una conexión sigue abierta tras 3 segundos)
        config.setLeakDetectionThreshold(3000);

        try (HikariDataSource ds = new HikariDataSource(config)) {
            System.out.println("[POOL INICIADO] Obteniendo conexión y provocando fuga intencionada...");

            // CONEXIÓN QUE NUNCA SE CIERRA (Leak intencionado)
            Connection connFugada = ds.getConnection();
            PreparedStatement ps = connFugada.prepareStatement("SELECT 1");
            ResultSet rs = ps.executeQuery();
            if (rs.next()) System.out.println("Consulta ejecutada con éxito en la conexión fugada.");

            System.out.println("Esperando 5 segundos para que HikariCP active la alarma de fuga...");
            Thread.sleep(5000);

            // Al cerrar la aplicación cerramos la conexión para no dejar residuos en MySQL
            connFugada.close();
            System.out.println("[FIN PRUEBA]");
        }
    }
}
```

Ejercicio 2.4: Capa de Persistencia Completa DAO/DTO con Transacciones Inyectadas

60 MIN

Diseña una capa DAO para la entidad `Cliente (id, nif, nombre, activo)` y `Factura (id, id_cliente, total, fecha)`. Implementa un método de servicio `emitirFactura(ClienteDTO cliente, BigDecimal total)` que en una sola transacción cree el cliente si no existe y registre su primera factura utilizando el DataSource de HikariCP.

Tablas necesarias (MySQL, motor InnoDB; `facturas.id` se autogenera con `AUTO_INCREMENT`):

```
CREATE TABLE clientes (
    id     BIGINT       NOT NULL PRIMARY KEY,
    nif    VARCHAR(9)   NULL,
    nombre VARCHAR(100) NOT NULL,
    activo CHAR(1)      NOT NULL DEFAULT 'S'
) ENGINE=InnoDB;

CREATE TABLE facturas (
    id         BIGINT        NOT NULL AUTO_INCREMENT PRIMARY KEY,
    id_cliente BIGINT        NOT NULL,
    total      DECIMAL(10,2) NOT NULL,
    fecha      DATE          NOT NULL,
    FOREIGN KEY (id_cliente) REFERENCES clientes(id)
) ENGINE=InnoDB;
```

🔑 SOLUCIÓN COMPLETA Y CÓDIGO FUNCIONAL:

```
package es.cifpaviles.dam2.ad.ut2.b2;

import javax.sql.DataSource;
import java.math.BigDecimal;
import java.sql.*;
import java.time.LocalDate;

public class ServicioFacturacionTransaccional {
    private final DataSource dataSource;

    public ServicioFacturacionTransaccional(DataSource ds) {
        this.dataSource = ds;
    }

    public void emitirFactura(long idCliente, String nombreCliente, BigDecimal totalFactura) {
        String sqlVerificarCliente = "SELECT id FROM clientes WHERE id = ?";
        String sqlCrearCliente = "INSERT INTO clientes (id, nombre, activo) VALUES (?, ?, 'S')";
        String sqlCrearFactura = "INSERT INTO facturas (id_cliente, total, fecha) "
                               + "VALUES (?, ?, ?)";

        try (Connection conn = dataSource.getConnection()) {
            conn.setAutoCommit(false);
            try {
                // 1. Verificar si cliente existe
                boolean existe = false;
                try (PreparedStatement ps = conn.prepareStatement(sqlVerificarCliente)) {
                    ps.setLong(1, idCliente);
                    try (ResultSet rs = ps.executeQuery()) {
                        existe = rs.next();
                    }
                }

                // Si no existe, crearlo
                if (!existe) {
                    try (PreparedStatement psIns = conn.prepareStatement(sqlCrearCliente)) {
                        psIns.setLong(1, idCliente);
                        psIns.setString(2, nombreCliente);
                        psIns.executeUpdate();
                        System.out.println("[INFO] Cliente dado de alta en la misma transacción: " + nombreCliente);
                    }
```

```
                }

                // 2. Crear factura
                try (PreparedStatement psFact = conn.prepareStatement(sqlCrearFactura)) {
                    psFact.setLong(1, idCliente);
                    psFact.setBigDecimal(2, totalFactura);
                    psFact.setObject(3, LocalDate.now());
                    psFact.executeUpdate();
                    System.out.println("[INFO] Factura emitida por importe de " + totalFactura + " €");
                }

                conn.commit();
                System.out.println("[OK] Transacción completada con éxito.");

            } catch (Exception e) {
                conn.rollback();
                throw new RuntimeException("Error en facturación. Rollback ejecutado.", e);
            } finally {
                conn.setAutoCommit(true);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```